Unit:-02 : Java Language basics

Topics List :

- Variables and Scope of Variables .
- Data Types and its types including Primitive and reference data type.
- Type Casting.
- Wrapper classes , Autoboxing and Unboxing .
- Operators and Types of Operator including Arithmetic , relational , logical etc.
- Operator Precedence.
- If-statement , if-else statement , nested if statement and switch statement .
- Loops and the types of loops like for loop , while loop, do-while loop.

Topic:-01
" Variables and scope of variables"

- **A variable** is a container that holds data during the execution of a program. Every variable must be declared with a specific data type, which determines the size and layout of the variable's memory.

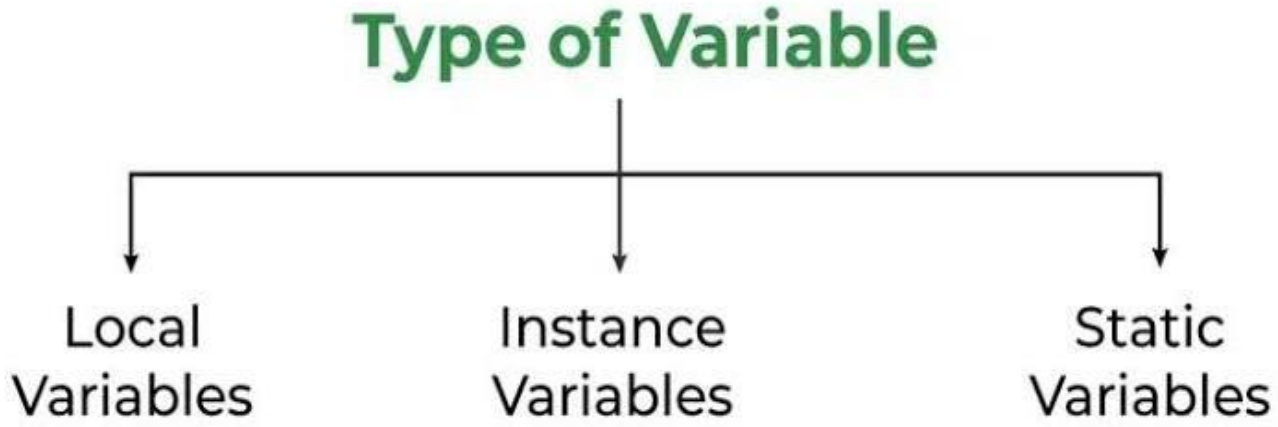
A variable in Java has three components :

- **Data Type:** Defines the kind of data stored (e.g., int, String, float).
- **Variable Name:** A unique identifier following Java naming rules.
- **Value:** The actual data assigned to the variable.

Syntax of declaration and initialization :

dataType variable Name = value;

Types of Variables :-



Local Variable : A variable defined within a block, method, or constructor is referred to as a local variable.

- The Local variable is created at the time of declaration and destroyed when the function completes its execution.
- The scope of local variables exists only within the block in which they are declared.
- We first need to initialize a local variable before using it within its scope.

Instance Variable : Instance variables are known as non-static variables and are declared in a class outside of any method, constructor, or

block. Instance variables are created when an object is instantiated and destroyed when the object is destroyed.

- Can have access specifiers; default access is used if none is specified.
- Accessed only through objects of the class.
- Instance Variables can be initialized using constructors or instance blocks.

Static Variable : Static variables in Java are variables declared with the static keyword inside a class but outside any method. They are

shared among all objects of the class and exist for the entire lifetime of the program.

- There is only one copy of a static variable for the entire class, and all objects share it
- Static variable are created at program start and destroyed when the program ends.
- Can be initialized using static blocks.

Scopes of Variables :

- Variable scope in Java defines the region of a program where a variable can be accessed and used. Java follows lexical (static) scoping, which means the scope of a variable is determined at compile time based on where the variable is declared.
- Scope determines where a variable is accessible in a program.
- Variables declared inside a block { } are limited to that block only.

Types of variable scope in java :

- Instance Variable Scope (Class Level Scope).
- Static Variable Scope (Class Level Scope).
- Local Variable Scope (Method Level Scope).
- Parameter Scope.
- Block Level Scope.

Instance Variable Scope (Class level Scope) :

Instance variables are non-static variables declared inside a class but outside methods, constructors, or blocks. Each object of the class gets its own separate copy of these variables.

For Example :

```
public class Instance Var Scope {
    int a;
    private String b;

    void method1() {....}
    int method2() {....}

    char c;
}
```

Static Variable scope (Class level Scope) :

Static variables are declared using the static keyword. Only one copy of the variable exists, and it is shared among all objects of the class.

For Example :

```
import java.io.*;


class Static Var Scope
{
    // static variable in Test class
    static int var = 10;
}
class main
{
   public static void main (String[] args)
 {
       // accessing the static variable
       System.out.println("Static Variable : "+Test.var);
   }
}
```

Local Variable Scope (Method level Scope) :

Local variables are declared inside methods, constructors, or blocks and can only be used within that method or block.

For Example :

```
public class Local Var Scope
{
    void method1()
    {
        // Local variable (Method level scope)
        int x;
    }
}
```

Parameter Scope :

Parameters are variables passed to methods or constructors. Their scope is limited to that specific method or constructor.

For Example :

```
class Parameter scope
 {
    private int x;


    public void setX(int x)
{
        this.x = x;
    }
}
```

Block Level Scope :

A variable declared inside pair of brackets "{" and "}" in a method has scope within the brackets only.

For Example :

```
public class Block Level Scope {
   public static void main(String args[]) {
       // Block Level Scope
      {
         // The variable x has scope within
         // brackets
         int x = 10;
         System.out.println(x);
      }
   }
}
```

Topic :-02
" Data Types and its types including Primitive and Reference Data types"

Data types specify the size and type of values that can be stored in a variable. Since Java is a **statically-typed language**, every variable must be declared with a data type before it can be used.

Syntax :

datatype variable Name;

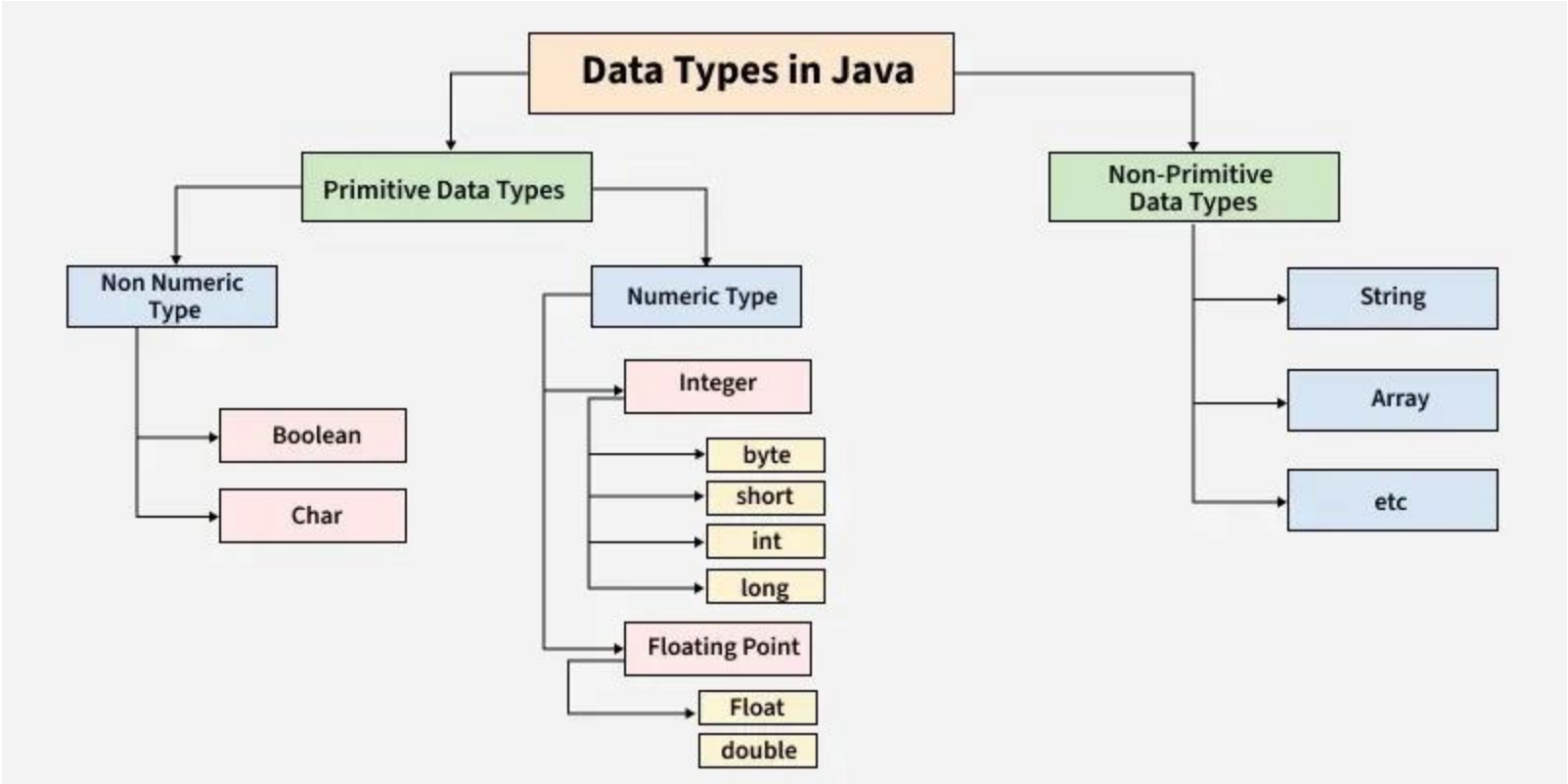
For Example :

Int num =120;

- Memory allocation determines how much memory is required for each variable
- Operations support defines what operations can be performed on data
- Each data type has a default value when not initialized

There Are Mainly two types of data types :

- Primitive Data Types
- Non-primitive Data Types (Reference Data Types).



➢ Primitive data types :

Primitive data types are the most basic data types built into the Java language. They serve as the building blocks for data manipulation and represent single, raw values. There are exactly **8 primitive data types** in Java.

➢ Non – Numeric Data Types :

Non – Numeric Data types is type of Primitive data types which only deals with string values such as "JAVA"

Again Non – Numeric Data types are broadly categorized into two types :

➢ **Boolean Data type :** Represents one of two logical values: true or false. It is commonly used in conditions and control statements.

Syntax : boolean booleanVar;

For Example : boolean isJavaFun = True;

➢ **Char Data type :** A 16-bit Unicode character used to store single symbols or letters.

Syntax : Char Char Var;

For Example : Char grade = 'A' ;

➢ Numeric Data Types :

Numeric Data types is type of Primitive data types which only deals with numbers such as ' 1 , 2, 3, 4 ' etc.

Numeric Data types are divided into following parts :

- **Numeric Data Types (Integer) :**
- **Byte Data Type :** An 8-bit signed integer used to save memory in large numeric arrays

Syntax : byte byte Var ;

For Example : byte age =25;

- **Short Data Type :** A 16-bit signed integer often used when memory is limited and values are moderate in size.

Syntax : short shortVar;

For Example : short students = 1000;

- **Int Data Type :** A 32-bit signed integer and the most commonly used numeric data type.

Syntax : int intVar;

For Example : int num = 2000;

- **Long Data Type :** A 64-bit signed integer used when int is not sufficient for large values.

Syntax : Long LongVar;

For Example : Long WorldPopulation = 78000000000000;

- **Numeric Data Types (Floating Point):**
- **Float Data Type :** A 32-bit single-precision floating-point type used for fractional values.

Syntax: float floatVar;

For Example: float pi = 3.14f;

- **Non – Primitive data types :**

Non – Primitive data types are also known as "reference data type ". Non-primitive data types are created by the programmer and are not predefined by Java (except for String). They are called **reference types** because they store references or memory addresses of the objects they point to, rather than the value itself.

- **String Data Type :** String represents a sequence of characters enclosed in double quotes. Unlike C/C++, Java strings are objects and are immutable.

Syntax : String str =" Hello ";

- **Array Data Type :** An array stores multiple elements of the same type in a single structure. Java arrays are objects, dynamically allocated, and indexed from 0.

Syntax : arr [ ] arr Name = { } ;

For example : int[] numbers = { 1 , 2 , 3 , 4 , 5 } ;

Data Type	Size	Default Value	Example
Boolean	1 bit	false	boolean isJavaFun = true;
byte	1 byte	0	byte age = 25;
short	2 bytes	0	short year = 2026;
int	4 bytes	0	int salary = 50000;
long	8 bytes	0L	long distance = 9876543210L;
float	4 bytes	0.0f	float pi = 3.14f;
double	8 bytes	0.0d	double price = 89.99;
char	2 bytes	'\u0000'	char grade = 'A';

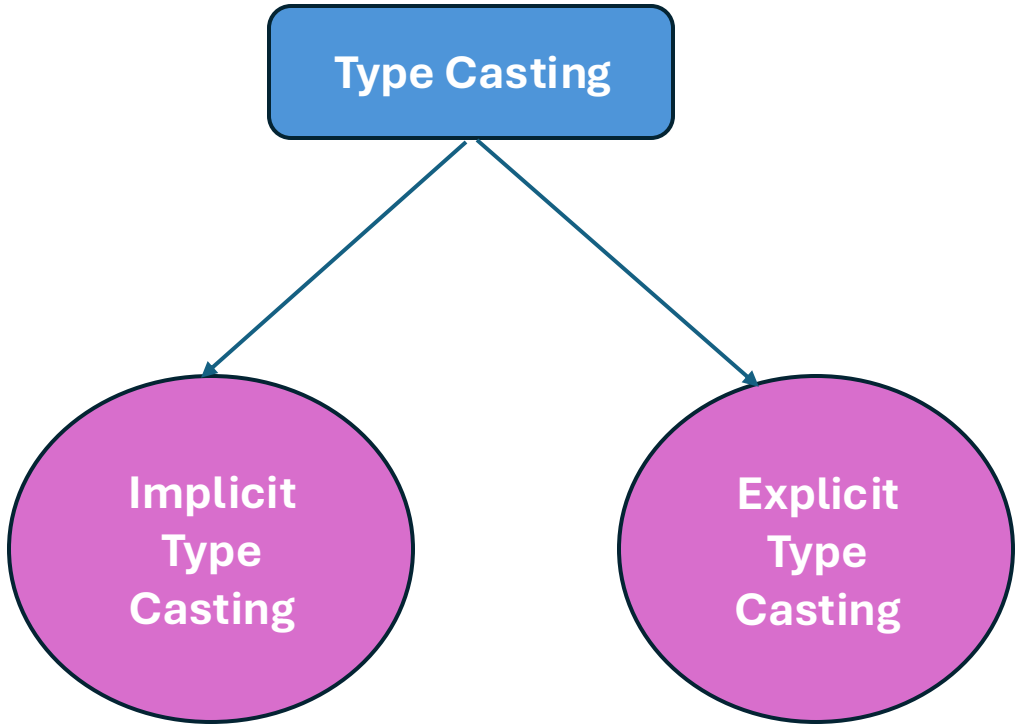
Topic :- 03
" Type Casting "

Type casting is the process of converting a variable from one data type to another.

Java is a statically typed language, it is very strict about compatibility between types.

There are Mainly Two Types of Types Casting :

- Implicit Type Casting (Widening).
- Explicit Type Casting (Narrowing).



Implicit Type Casting (widening) :

Implicit Type Casting is a kind of type casting which works automatically . In the sense the type of data has been automatically converted . It is also known as widening. Implicit Type Casting works Smaller data type to Larger data types .

Direction : byte ->short -> char -> int -> long -> float -> double .

For Example :

Int num = 25;

Double d = num ;

Explicit Type Casting (Narrowing) :

Explicit Type Casting is a Kind of type casting which works manually . In the sense the type of data has been manually converted. It is also known as Narrowing . Explicit Type Casting Works Larger data types to smaller data types .

Direction : double -> float -> long -> int -> char -> short -> byte.

For Example :

Double d = 102.3;

Int num = (num) d;

Topic :- 04
" Wrapper classes , Autoboxing and Unboxing "

In Java, the type system is split into two categories: **primitive types** (like `int`, `char`, `double`) which are fast and lightweight, and **objects** (which have methods and can be used with Java's Collections framework). To bridge the gap between these two worlds, Java uses **Wrapper Classes**, **Autoboxing**, and **Unboxing**.

➢ **Wrapper Classes** :- A **Wrapper Class** is a class whose object wraps or contains a primitive data type. Think of it as putting a primitive value inside an object "envelope."

Why Do we need them ?

➢ **Collections Framework:** Java collections (like `Array List`, `HashMap`) can only store *objects*, not primitives. You can't have an `Array List<int>`, but you can have an `Array List<Integer>`.

Every primitive type has a corresponding wrapper class in the `java.lang` package:

Primitive Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

➢ Autoboxing :

Autoboxing is the automatic conversion that the Java compiler makes between primitive types and their corresponding object wrapper classes.

Instead of writing explicit conversion code, the compiler does it for you behind the scenes.

➢ **Manual boxing (Old way):** `Integer num = Integer.valueOf(100);`

➢ **Autoboxing (Modern way):** `Integer num = 100;`

For Example :

```
ArrayList<Integer> list = new ArrayList<>();
// Autoboxing: The primitive int '25' is automatically converted to an Integer object
list.add(25);
```

Unboxing :

Unboxing is the exact reverse process. It is the automatic conversion of a wrapper object back into its corresponding primitive type.

➢ **Manual unboxing (Old way):** `int p = num.intValue();`

➢ **Unboxing (Modern way):** `int p = num;`

For Example :

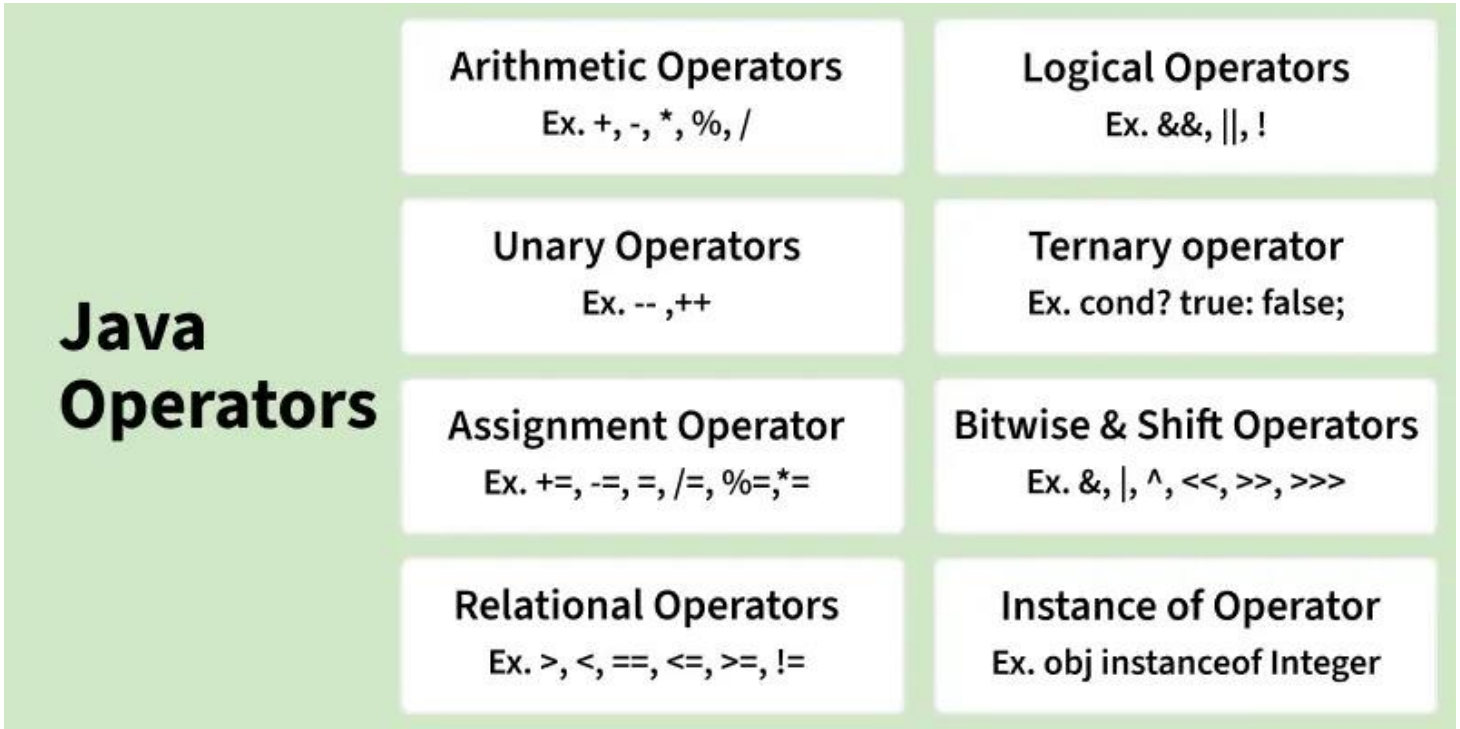
```
Integer wrapperNum = Integer.valueOf(50);

// Unboxing: The Integer object is automatically converted back  to a primitive int
int primitiveNum = wrapperNum;

// Unboxing also happens during arithmetic operations
int sum = wrapperNum + 10; // wrapperNum is unboxed to add it to 10
```

Topic :-05
" Operators and Types of Operator "

In Java, **operators** are special symbols used to perform operations on variables and values. Think of them as the verbs of the programming language—they make things happen.



- **Arithmetic Operators :-**

Arithmetic Operators are used to perform arithmetic operations on primitive numeric data types such as int, float, and double.

Arithmetic Operators includes operation like :

- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Division (/)
- Modulus (%)

Program :

```
Public class arithmeticoperator
{
  Public static void main(String args[])
  {
    Int a =10;
    Int b =20;
   System.out.println("Addition of two number is = " +(a+b));
   System.out.println("Subtraction of two number is = " +(a-b));
   System.out.println("Multiplication of two number is = " +(a*b));
   System.out.println("Division of two number is = " +(a/b));
   System.out.println("Modulus of two number is = " +(a%b));
   }
}
```

- **Unary Operators :-**

Unary Operators need only one operand. They are used to increment, decrement, or negate a value. There is Two thing either operator perform Post increment or pre increment and post decrement or pre decrement .

Unary Operator Includes Operations Like :

- Post Increment (a++) .
- Pre Increment (++a).
- Post Decrement (b--).
- Pre Decrement (--b).

Program :

```
Public class unaryoperator
{
    public static void main(String args[])
    {
     Int a =10;
     Int b = 10;

     System.out.println("post Increment : " + (a++));
     System.out.println("Pre Increment : " + (++a));
     System.out.println("Post decrement : " + (b--));
     System.out.println("Pre decrement : " + (--b));
     }
}
```

- **Assignment Operator :-**

The Assignment Operator assigns a value from the right-hand side to a variable on the left. Since it has right-to-left associativity, the right-hand value must be declared or constant. The most common one is =, but Java also has **compound assignment operators** that combine an arithmetic operation with an assignment.

Assignment Operator includes operation like :

Simple assignment (=).

Add and Assign (+=).

Subtract and Assign (- =).

Multiply and Assign (* =).

Divide and Assign (/ =) .

Modulus and Assign (% =).

Program :

```
Public class assignmentoperator
{
    public static void main(String args[])
    {
      int num = 10 ;
      System.out.println("Initial Value : " +num);
      num += 5;
      Sytem.out.println(" Add and Assign : " +num);
      num -=5;
      System.out.println("Subtract and Assign : " +num);
      num *=5;
      System.out.println("multiply and Assign : " +num);
      Num /=4;
      System.out.println("Divide and Assign : " +num );
      Num %=3;
     Sytem.out.println(" Modulus and Assign : " +num );
    }
}
```

- **Relational Operator :**

Relational Operator are used to check for relations like equality, greater than, and less than. They return Boolean results After the comparison and are extensively used in looping statements as well as conditional if-else statements.

Relational operator includes Operations like :

Equals to (==).

Not Equals to (!=).

Greater than (>).

Less than (<).

Greater than or equal to (>=).

Less than or equal to (<=).

Program :

```
Public class Relationaloperator
{
  Public static void main(String args[])
  {
    Int a = 10;
    Int b = 30;
    System.out.println(" Equals to : " + (a==b));
    System.out.println(" Not Equals to : " + (a!=b));
    System.out.println(" Greater than : " + (a>b));
    System.out.println(" Less than : " + (a<b));
    System.out.println(" Greater than or Equals to : " + (a>=b));
    System.out.println(" Less than or Equals to : " + (a<=b));
    }
}
```

- **Logical Operator :**

Logical Operator are used to perform "logical AND" and "logical OR" operations, similar to AND gate and OR gate in digital electronics. They have a short-circuiting effect, meaning the second condition is not evaluated if the first is false.

Logical Operators includes Operation like :

Logical AND (&&) - Returns True only if both conditions are true .

Logical OR(||) - Returns True if At least one condition is true .

Logical NOT (!) - Inverse the result .

Program :

```
Public class logicaloperator
{
  Public static void main(String args[])
   {
     Boolean x = true;
     Boolean y = false;

      System.out.println(" x&&y : " + (x&&y));
      System.out.println("x||y : " +(x||y));
      System.out.println("!x : " + (!x));
     }
}
```

- **Ternary Operator :**

The ternary operator is a shorthand version of an `if-then-else` statement. It takes three operands.

Syntax: `variable = (condition) ? expressionTrue : expressionFalse;`

It uses two symbols:

- **? (The Question):** This separates the condition from the code that runs if the condition is true. Think of it as asking, "Is this condition true?"
- **: (The Otherwise / Else):** This separates the "true result" from the "false result". Think of it as saying, "Otherwise, do this."

Program :

```
Public class Ternaryoperator
{
  Public static void main(String args[])
  {
    Int a =20 , b =10 , c = 30 , Result ;
    result = (a>b) ? (a>c) ? A : c  : (b>c) ? B : c );
    System.out.println("maximum of three number is =  " +result );
   }
}
```

- **Bitwise and Shift Operator :-**
- **Bitwise Operator -** Bitwise operators compare numbers bit-by-bit. It Includes :
  - **Bitwise AND(&) -** Compares each bit. It returns 1 **only if both bits are 1**.
  - **Bitwise OR(|) -** Returns 1 if **at least one** of the bits is 1.
  - **Bitwise XOR(^) -** Returns 1 if the bits are **different** (one is 0 and the other is 1). If they are the same, it returns 0.
  - **Bitwise Complement(~)-** This is a unary operator (takes only one input). It **flips all the bits** (0 becomes 1, 1 becomes 0).
- **Shift Operator -** Shift operators literally slide the binary bits of a number to the left or right. It Includes :
  - **Left Shift (<<) -** Shifts the bits to the left by a specified number of positions and fills the empty spaces on the right with 0.
  - **Signed Right Shift (>>) -** Shifts bits to the right. The empty spaces on the left are filled depending on the original number's sign bit (0 for positive, 1 for negative) to preserve the positive/negative status of the number.
  - **Unsigned Right Shift (>>>) -** Shifts bits to the right, but **always fills the empty spaces on the left with 0**, regardless of whether the number was originally positive or negative. (For positive numbers, >> and >>> behave identically).

Program :

```
Public class BSO
{
      Public static void main(String args[])
      {
        Int a = 5;
        Int b = 3;
        //---Bitwise Operator
        System.out.println ("a&b = " +(a&b));
        System.out.println ("a|b = " +(a|b));
      System.out.println ("a^b = " +(a^b));
      //---shift Operator
     System.out.println ( " a << 1 = " +(a<<1));
     System.out.println ( " a >> 1 = " +(a>>1));
    }
}
```

- **Instance of Operator :**

The Instance of Operator is Comparison Operator Used Specifically Object References. It checks whether an object belongs to a specific class or implemented a specific interface. It returns either True or False .

Program :

```
 class user ()
class Admin () extends user()
Public class main ()
{
   Public static void main(String args[])
   {
     User regularUser = new User();
     User adminuser = new Admin();
     System.out.println (adminuser instance of Admin);
     System.out.println (adminuser instance of user);
     System.out.println (regularuser instance of Admin);
   }
}
```

Topic –06
"Operator Precedence "

Operator precedence specifies the order in which different operators are evaluated in an expression. Operators with higher precedence are evaluated before operators with lower precedence, regardless of their position in the expression.

- Higher-precedence operators are evaluated before lower-precedence operators.
- Parentheses () can be used to override the default precedence.
- Helps avoid ambiguity in complex expressions

Syntax : -

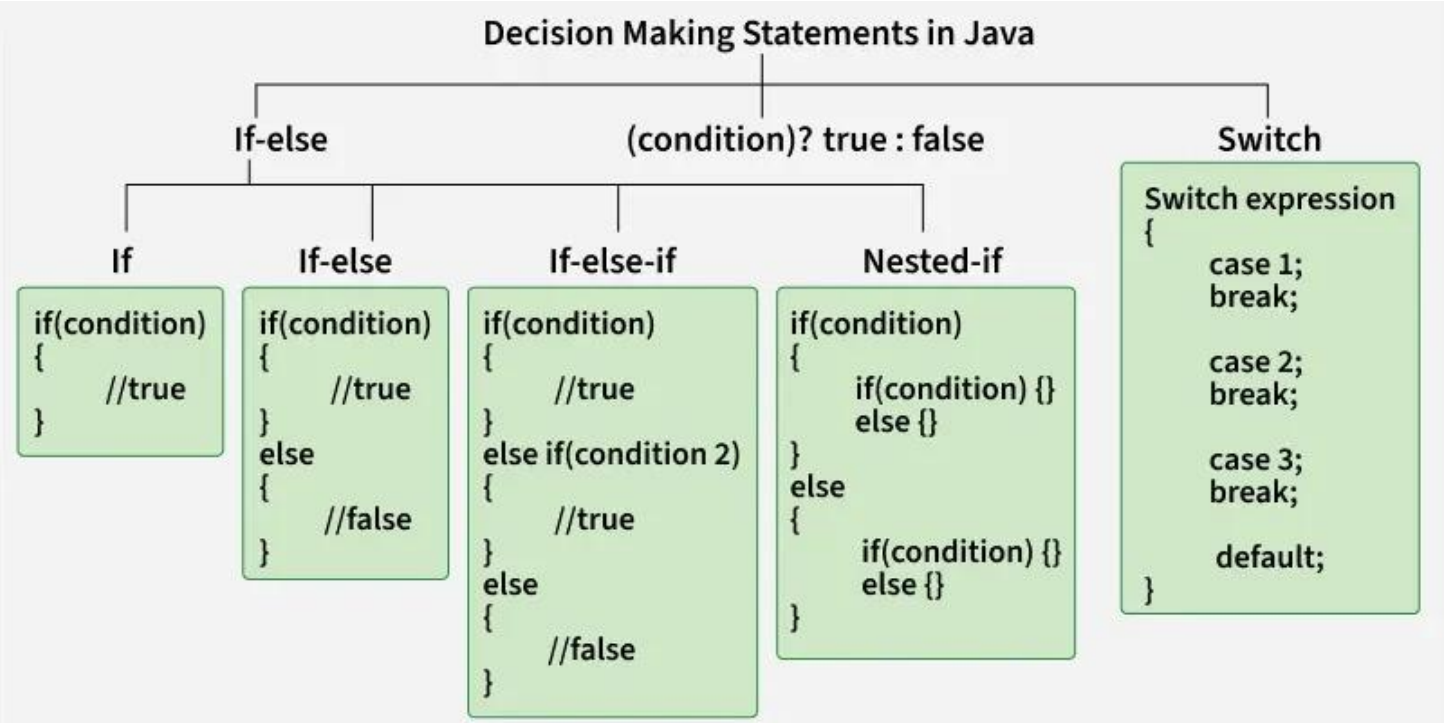
operand1 operator operand2

Operator's Precedence in Java

Operators	Precedence
!, +, - (unary Operators)	First (Highest)
* , / , %	Second
+ , -	Third
< , <= , >=, >	Fourth
== , !=	Fifth
&&	Sixth
||	Seventh
= (assignment Operator)	Lowest

Malik AB

Topic :- 07 "IF Statement , IF- Else Statement , Nested IF-Else , Switch Case Statement "



Decision-making statements in Java allow a program to choose different execution paths based on specified conditions. They help control the flow of a program by executing certain blocks of code only when particular conditions are true.

- **IF Statement :** The IF Statement is the simplest decision – making statement . It execute the block of code only the given condition is true .

For Example :

```
Public class IF
{
 Public static void main(String[] args)
  {
   Int num = 10;
   If (num < 20 )
   {
    System.out.println ("Given Condition is true");
   }
}
}
```

- **IF – ELSE Statement :** The IF – ELSE Statement allow us to execute the one block of code only when then condition of true otherwise it execute the else block in case of false condition .

- For Example :

```
Public class IF-ELSE
{
   Public static void main(String[] args)
 {
     Int num = 10;
     If (num < 20 )
     {
         System.out.println ("Given Condition is true");
     }
      Else
     {
       System.ou.println("Given condition is false " );
     }
 }
}
```

- **Nested IF Statements :** In Nested IF Statements an IF Statement is Inside an another IF Statements it is useful when the second condition depends on first condition .

For Example :

```
Public class Nested IF
{
   Public static void main[String[] args)
{
    Int num = 20;
    IF(num > 5 )
{
     System.out.println(" num is greater than 5 ");
       IF(num<10)
       {
         System.out.println("num is lesser than 10 " );
       }
 }
}
}
```

- **IF-ELSE-IF Statement :** In this kind of decision-making statement there are multiple independent conditions to be checked in order . As soon As one condition is true , its block executes rest all block has been skipped .

For Example :

```
Public class IF ELSE IF
{
    public static void main(String[] args)
    {
        int  I = 10;
        if(I ==9)
        {
            System.out.println(" I is 9 ");
        }
        else if(I ==8)
        {
            System.out.println(" I is 8 ");
        }
```

```
else if(I==10)
        {
            System.out.println("I is 10");
        }
        else
        {
            System.out.println(" I is not present ");
        }
    }
}
```

- **Switch-case statement :-**The Switch-case statement is multiway branch statement. It provide easy way to dispatch execution of different parts of code based on value of expression . In which **switch** and **break** keyword used to break the case and switch on the next case .

- **For Example:-**

```
Public class switchcase
{
    public static void main(String[] args)
    {
        int num =10;
        switch (num) {
        case 5:
            System.out.println("It is 5");
            break;
        case 10;
            System.out.println("It is 10");
            break;
        case 15;
            System.out.println("It is 15");
            break;
        default :
        System.out.println("Not Present");
        }
    }
```

Topic :-08
"Loops and types of loops like for loop , while loop , do-while loop

In Programming a Loop is Control structure used to repeat a specific block of code multiple times until a certain condition is meet. Instead of writing the same code 100, 1000 times you use the Loop for automation .

- There are three primary types of loop as follows :
- FOR Loop :

The for loop is an **entry-controlled loop** best used when you **know in advance exactly how many times** you want to execute a block of code.

For Example -

```
Public class Forloop
{
    public static void main(String[] args)
    {
        for(int i = 0; i<=10; i++)
        {
            System.out.println("For Loop Count : " +i);
        }
    }
}
```

- **While Loop :**

While Loop is also one of the control strucutre . Use this when you want to loop based on a condition, and you aren't 100% sure how many iterations it will take.

➢ For Example :

```
 Public class whileloop
{
    public static void main(String[] args)
    {
     Int count = 1;
     While(count <= 5)
     {
      System.out.println("While Loop count : " +count);
      Count++;
     }
  }
}
```

➢ **Do – While Loop :**

The Do-While Loop ensures that the block of code is executes at least once before checking the condition . Use this when you we want the code inside the loop to execute **at least once**

For Example:

```
public class DoWhileLoop
{
    public static void main(String[] args)
    {
      int count = 1;
      do {
         System.out.println("Do-While Loop Count: " + count);
        count++;
        } while (count <= 5);
     }
}
```